# ML/AI TOOLS & OPERATIONS GUIDE

## AI-Based Monitoring Solution for Oil & Gas Industry

**Document Version**: 1.0 **Date**: December 2025 **Audience**: ML Engineers, Data Scientists, Developers

---

# TABLE OF CONTENTS


1. ML Development Environment
2. PyTorch Operations
3. MLflow Experiment Tracking
4. Model Training Workflows
5. TensorRT Optimization
6. ONNX Export & Conversion
7. Model Evaluation Tools
8. RAG Pipeline Tools
9. Model Deployment
10. Complete ML Workflows


---

# 1. ML DEVELOPMENT ENVIRONMENT

## 1.1 Environment Setup

```
# Create conda environment
conda create -n oilgas-ml python=3.10
conda activate oilgas-ml

# Install PyTorch with CUDA
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# Install ML libraries
pip install \
    numpy pandas scikit-learn scipy \
    mlflow optuna wandb \
    transformers sentence-transformers \
    onnx onnxruntime-gpu \
    tensorrt

# Install monitoring tools
pip install \
    loguru tqdm rich \
    matplotlib seaborn plotly

# Verify GPU
python -c "import torch; print(f'CUDA: {torch.cuda.is_available()}, Devices: {torch.cuda.device_count()}
```

## 1.2 Project Structure

```
ml/
  configs/                    # Training configurations
```

```
    anomaly_detection.yaml
    rul_prediction.yaml
    ppe_detection.yaml
data/                       # Data processing
    datasets.py
    transforms.py
    loaders.py
models/                     # Model definitions
    anomaly/
        autoencoder.py
        isolation_forest.py
    rul/
        cnn_lstm.py
        transformer.py
    vision/
        yolo_ppe.py
    rag/
        embeddings.py
        retriever.py
training/                   # Training scripts
    train_anomaly.py
    train_rul.py
    train_ppe.py
evaluation/                 # Evaluation scripts
    evaluate.py
    metrics.py
export/                     # Model export
    to_onnx.py
    to_tensorrt.py
inference/                  # Inference scripts
    predict.py
    batch_inference.py
utils/
    logging.py
    visualization.py
```

### 1.3 Configuration Management

```yaml
# configs/anomaly_detection.yaml
model:
  name: "AnomalyAutoencoder"
  input_dim: 15
  latent_dim: 3
  hidden_dims: [10, 5]
  dropout: 0.2
  activation: "relu"

data:
  train_path: "data/training/vibration_features.parquet"
  val_split: 0.15
  test_split: 0.15
  batch_size: 256
  num_workers: 4
```

```yaml
training:
  epochs: 100
  learning_rate: 0.001
  weight_decay: 0.0001
  scheduler: "cosine"
  early_stopping:
    patience: 10
    min_delta: 0.0001

mlflow:
  experiment_name: "anomaly_detection"
  tracking_uri: "http://mlflow.oilgas.internal:5000"
  artifact_location: "s3://mlflow-artifacts/"

hardware:
  device: "cuda"
  mixed_precision: true
  distributed: false
```

```python
# utils/config.py
import yaml
from pathlib import Path
from dataclasses import dataclass
from typing import Dict, Any, Optional


@dataclass
class ModelConfig:
    name: str
    input_dim: int
    latent_dim: int
    hidden_dims: list
    dropout: float = 0.2
    activation: str = "relu"


@dataclass
class TrainingConfig:
    epochs: int
    learning_rate: float
    weight_decay: float
    scheduler: str
    early_stopping: Dict


def load_config(config_path: str) -> Dict[str, Any]:
    """Load configuration from YAML file"""
    with open(config_path) as f:
        return yaml.safe_load(f)


def save_config(config: Dict, output_path: str):
    """Save configuration to YAML file"""
    with open(output_path, 'w') as f:
```

```
        yaml.dump(config, f, default_flow_style=False)
```

---

# 2. PYTORCH OPERATIONS

## 2.1 Model Definition

```
#!/usr/bin/env python3
# models/anomaly/autoencoder.py
"""
Anomaly Detection Autoencoder

Usage:
    from models.anomaly.autoencoder import AnomalyAutoencoder
    model = AnomalyAutoencoder(input_dim=15, latent_dim=3)
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import List, Tuple, Optional


class AnomalyAutoencoder(nn.Module):
    """
    Autoencoder for anomaly detection in sensor data.

    Architecture:
        Encoder: input_dim -> hidden_dims -> latent_dim
        Decoder: latent_dim -> hidden_dims (reversed) -> input_dim

    Anomaly score = reconstruction error (MSE)
    """

    def __init__(
        self,
        input_dim: int = 15,
        latent_dim: int = 3,
        hidden_dims: List[int] = [10, 5],
        dropout: float = 0.2,
        activation: str = "relu"
    ):
        super().__init__()

        self.input_dim = input_dim
        self.latent_dim = latent_dim
        self.hidden_dims = hidden_dims

        # Activation function
        activations = {
            "relu": nn.ReLU(),
            "leaky_relu": nn.LeakyReLU(0.2),
            "elu": nn.ELU(),
```

```
            "gelu": nn.GELU()
        }
        self.activation = activations.get(activation, nn.ReLU())

        # Build encoder
        encoder_layers = []
        prev_dim = input_dim
        for hidden_dim in hidden_dims:
            encoder_layers.extend([
                nn.Linear(prev_dim, hidden_dim),
                nn.BatchNorm1d(hidden_dim),
                self.activation,
                nn.Dropout(dropout)
            ])
            prev_dim = hidden_dim

        encoder_layers.append(nn.Linear(prev_dim, latent_dim))
        self.encoder = nn.Sequential(*encoder_layers)

        # Build decoder (mirror of encoder)
        decoder_layers = []
        prev_dim = latent_dim
        for hidden_dim in reversed(hidden_dims):
            decoder_layers.extend([
                nn.Linear(prev_dim, hidden_dim),
                nn.BatchNorm1d(hidden_dim),
                self.activation,
                nn.Dropout(dropout)
            ])
            prev_dim = hidden_dim

        decoder_layers.append(nn.Linear(prev_dim, input_dim))
        self.decoder = nn.Sequential(*decoder_layers)

        # Initialize weights
        self.apply(self._init_weights)

    def _init_weights(self, module):
        """Initialize weights using He initialization"""
        if isinstance(module, nn.Linear):
            nn.init.kaiming_normal_(module.weight, mode='fan_in', nonlinearity='relu')
            if module.bias is not None:
                nn.init.zeros_(module.bias)

    def encode(self, x: torch.Tensor) -> torch.Tensor:
        """Encode input to latent space"""
        return self.encoder(x)

    def decode(self, z: torch.Tensor) -> torch.Tensor:
        """Decode from latent space"""
        return self.decoder(z)

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """
```

```
        Forward pass

        Args:
            x: Input tensor [batch_size, input_dim]

        Returns:
            reconstructed: Reconstructed input
            latent: Latent representation
        """
        latent = self.encode(x)
        reconstructed = self.decode(latent)
        return reconstructed, latent

    def compute_anomaly_score(self, x: torch.Tensor) -> torch.Tensor:
        """
        Compute anomaly score (reconstruction error)

        Args:
            x: Input tensor [batch_size, input_dim]

        Returns:
            Anomaly scores [batch_size]
        """
        reconstructed, _ = self.forward(x)
        mse = F.mse_loss(reconstructed, x, reduction='none')
        return mse.mean(dim=1)

    def predict_anomaly(
        self,
        x: torch.Tensor,
        threshold: float = None
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Predict if samples are anomalies

        Args:
            x: Input tensor
            threshold: Anomaly threshold (if None, returns only scores)

        Returns:
            is_anomaly: Boolean tensor
            scores: Anomaly scores
        """
        scores = self.compute_anomaly_score(x)

        if threshold is not None:
            is_anomaly = scores > threshold
            return is_anomaly, scores

        return scores

    @torch.no_grad()
    def fit_threshold(
        self,
```

```
        dataloader,
        percentile: float = 95.0
    ) -> float:
        """
        Fit anomaly threshold from normal data

        Args:
            dataloader: DataLoader with normal samples
            percentile: Percentile for threshold

        Returns:
            threshold value
        """
        self.eval()
        all_scores = []

        for batch in dataloader:
            if isinstance(batch, (tuple, list)):
                x = batch[0]
            else:
                x = batch

            x = x.to(next(self.parameters()).device)
            scores = self.compute_anomaly_score(x)
            all_scores.append(scores.cpu())

        all_scores = torch.cat(all_scores)
        threshold = torch.quantile(all_scores, percentile / 100.0).item()

        return threshold


class CNNLSTM_RUL(nn.Module):
    """
    CNN-LSTM model for Remaining Useful Life prediction.

    Architecture:
        CNN: Extract local patterns from sensor windows
        LSTM: Capture temporal dependencies
        Attention: Focus on important time steps
        FC: Output RUL prediction
    """

    def __init__(
        self,
        input_channels: int = 15,
        seq_length: int = 50,
        cnn_filters: List[int] = [32, 64, 128],
        lstm_hidden: int = 128,
        lstm_layers: int = 2,
        attention_heads: int = 4,
        dropout: float = 0.3
    ):
        super().__init__()
```

```
        self.input_channels = input_channels
        self.seq_length = seq_length

        # CNN layers
        cnn_layers = []
        in_channels = input_channels
        for filters in cnn_filters:
            cnn_layers.extend([
                nn.Conv1d(in_channels, filters, kernel_size=3, padding=1),
                nn.BatchNorm1d(filters),
                nn.ReLU(),
                nn.MaxPool1d(2),
                nn.Dropout(dropout)
            ])
            in_channels = filters

        self.cnn = nn.Sequential(*cnn_layers)

        # Calculate CNN output size
        cnn_out_length = seq_length // (2 ** len(cnn_filters))
        self.cnn_out_dim = cnn_filters[-1] * cnn_out_length

        # LSTM
        self.lstm = nn.LSTM(
            input_size=cnn_filters[-1],
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            dropout=dropout if lstm_layers > 1 else 0,
            bidirectional=True
        )

        # Attention
        self.attention = nn.MultiheadAttention(
            embed_dim=lstm_hidden * 2,  # bidirectional
            num_heads=attention_heads,
            dropout=dropout,
            batch_first=True
        )

        # Output layers
        self.fc = nn.Sequential(
            nn.Linear(lstm_hidden * 2, 64),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(64, 1)
        )

        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            nn.init.kaiming_normal_(module.weight)
```

```
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.LSTM):
            for name, param in module.named_parameters():
                if 'weight' in name:
                    nn.init.orthogonal_(param)
                elif 'bias' in name:
                    nn.init.zeros_(param)

    def forward(
        self,
        x: torch.Tensor,
        return_attention: bool = False
    ) -> torch.Tensor:
        """
        Forward pass

        Args:
            x: Input tensor [batch, seq_length, features]
            return_attention: If True, return attention weights

        Returns:
            rul: Predicted RUL [batch, 1]
        """
        # x: [batch, seq, features] -> [batch, features, seq]
        x = x.permute(0, 2, 1)

        # CNN
        cnn_out = self.cnn(x)  # [batch, filters, reduced_seq]

        # Prepare for LSTM: [batch, seq, features]
        lstm_in = cnn_out.permute(0, 2, 1)

        # LSTM
        lstm_out, _ = self.lstm(lstm_in)  # [batch, seq, hidden*2]

        # Attention
        attn_out, attn_weights = self.attention(
            lstm_out, lstm_out, lstm_out
        )

        # Global average pooling
        pooled = attn_out.mean(dim=1)  # [batch, hidden*2]

        # Output
        rul = self.fc(pooled)

        if return_attention:
            return rul, attn_weights

        return rul

    @torch.no_grad()
    def predict_with_uncertainty(
```

```
        self,
        x: torch.Tensor,
        n_samples: int = 100
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        MC Dropout for uncertainty estimation

        Args:
            x: Input tensor
            n_samples: Number of MC samples

        Returns:
            mean: Mean prediction
            std: Standard deviation (uncertainty)
        """
        self.train()  # Enable dropout

        predictions = []
        for _ in range(n_samples):
            pred = self.forward(x)
            predictions.append(pred)

        predictions = torch.stack(predictions)
        mean = predictions.mean(dim=0)
        std = predictions.std(dim=0)

        self.eval()

        return mean, std
```

### 2.2 Data Loading

```
#!/usr/bin/env python3
# data/datasets.py
"""
Dataset classes for ML training

Usage:
    from data.datasets import SensorDataset, create_dataloaders
    train_loader, val_loader, test_loader = create_dataloaders(config)
"""

import torch
from torch.utils.data import Dataset, DataLoader, random_split
import pandas as pd
import numpy as np
from pathlib import Path
from typing import Tuple, Optional, List
from sklearn.preprocessing import StandardScaler, RobustScaler
import joblib
from loguru import logger


class SensorDataset(Dataset):
```

```
    """
    Dataset for sensor feature data.

    Loads preprocessed vibration features for anomaly detection.
    """

    def __init__(
        self,
        data_path: str,
        feature_columns: List[str] = None,
        label_column: str = None,
        scaler: Optional[StandardScaler] = None,
        fit_scaler: bool = False
    ):
        """
        Args:
            data_path: Path to parquet/csv file
            feature_columns: Columns to use as features (None = all numeric)
            label_column: Column with labels (None for unsupervised)
            scaler: Pre-fitted scaler (None to create new)
            fit_scaler: Whether to fit the scaler on this data
        """
        self.data_path = Path(data_path)

        # Load data
        if self.data_path.suffix == '.parquet':
            df = pd.read_parquet(data_path)
        else:
            df = pd.read_csv(data_path)

        logger.info(f"Loaded {len(df)} samples from {data_path}")

        # Select feature columns
        if feature_columns is None:
            feature_columns = df.select_dtypes(include=[np.number]).columns.tolist()
            if label_column and label_column in feature_columns:
                feature_columns.remove(label_column)

        self.feature_columns = feature_columns
        self.features = df[feature_columns].values.astype(np.float32)

        # Labels (optional)
        self.labels = None
        if label_column and label_column in df.columns:
            self.labels = df[label_column].values

        # Scaling
        self.scaler = scaler
        if fit_scaler or scaler is None:
            self.scaler = StandardScaler()
            self.features = self.scaler.fit_transform(self.features)
        elif scaler is not None:
            self.features = self.scaler.transform(self.features)
```

```
        self.features = torch.from_numpy(self.features).float()

        if self.labels is not None:
            self.labels = torch.from_numpy(self.labels)

    def __len__(self) -> int:
        return len(self.features)

    def __getitem__(self, idx: int):
        if self.labels is not None:
            return self.features[idx], self.labels[idx]
        return self.features[idx]

    def save_scaler(self, path: str):
        """Save fitted scaler"""
        joblib.dump(self.scaler, path)

    @classmethod
    def load_scaler(cls, path: str) -> StandardScaler:
        """Load fitted scaler"""
        return joblib.load(path)


class SequenceDataset(Dataset):
    """
    Dataset for sequential sensor data (RUL prediction).

    Creates sequences with sliding window approach.
    """

    def __init__(
        self,
        data_path: str,
        sequence_length: int = 50,
        stride: int = 1,
        feature_columns: List[str] = None,
        target_column: str = 'rul',
        asset_column: str = 'asset_id',
        scaler: Optional[StandardScaler] = None,
        fit_scaler: bool = False
    ):
        self.sequence_length = sequence_length
        self.stride = stride

        # Load data
        if data_path.endswith('.parquet'):
            df = pd.read_parquet(data_path)
        else:
            df = pd.read_csv(data_path)

        # Select features
        if feature_columns is None:
            exclude = [target_column, asset_column, 'timestamp', '_time']
            feature_columns = [c for c in df.select_dtypes(include=[np.number]).columns
```

```
                            if c not in exclude]

        self.feature_columns = feature_columns
        self.sequences = []
        self.targets = []

        # Process each asset separately
        for asset_id in df[asset_column].unique():
            asset_df = df[df[asset_column] == asset_id].sort_values('timestamp')

            features = asset_df[feature_columns].values.astype(np.float32)
            target = asset_df[target_column].values.astype(np.float32)

            # Create sequences
            for i in range(0, len(features) - sequence_length + 1, stride):
                seq = features[i:i + sequence_length]
                tgt = target[i + sequence_length - 1]

                self.sequences.append(seq)
                self.targets.append(tgt)

        self.sequences = np.array(self.sequences)
        self.targets = np.array(self.targets)

        logger.info(f"Created {len(self.sequences)} sequences")

        # Scaling
        self.scaler = scaler
        if fit_scaler or scaler is None:
            self.scaler = StandardScaler()
            # Reshape for fitting: [n_samples * seq_length, features]
            flat_sequences = self.sequences.reshape(-1, self.sequences.shape[-1])
            self.scaler.fit(flat_sequences)

        # Apply scaling
        flat_sequences = self.sequences.reshape(-1, self.sequences.shape[-1])
        scaled = self.scaler.transform(flat_sequences)
        self.sequences = scaled.reshape(self.sequences.shape)

        self.sequences = torch.from_numpy(self.sequences).float()
        self.targets = torch.from_numpy(self.targets).float()

    def __len__(self) -> int:
        return len(self.sequences)

    def __getitem__(self, idx: int):
        return self.sequences[idx], self.targets[idx]


def create_dataloaders(
    config: dict,
    dataset_class: type = SensorDataset
) -> Tuple[DataLoader, DataLoader, DataLoader]:
    """
```

```
    Create train/val/test dataloaders from config.

    Args:
        config: Configuration dictionary
        dataset_class: Dataset class to use

    Returns:
        train_loader, val_loader, test_loader
    """
    data_config = config['data']

    # Load full dataset
    full_dataset = dataset_class(
        data_path=data_config['train_path'],
        fit_scaler=True
    )

    # Split dataset
    total_size = len(full_dataset)
    val_size = int(total_size * data_config['val_split'])
    test_size = int(total_size * data_config['test_split'])
    train_size = total_size - val_size - test_size

    train_dataset, val_dataset, test_dataset = random_split(
        full_dataset,
        [train_size, val_size, test_size],
        generator=torch.Generator().manual_seed(42)
    )

    logger.info(f"Split sizes - Train: {train_size}, Val: {val_size}, Test: {test_size}")

    # Create dataloaders
    train_loader = DataLoader(
        train_dataset,
        batch_size=data_config['batch_size'],
        shuffle=True,
        num_workers=data_config.get('num_workers', 4),
        pin_memory=True
    )

    val_loader = DataLoader(
        val_dataset,
        batch_size=data_config['batch_size'],
        shuffle=False,
        num_workers=data_config.get('num_workers', 4),
        pin_memory=True
    )

    test_loader = DataLoader(
        test_dataset,
        batch_size=data_config['batch_size'],
        shuffle=False,
        num_workers=data_config.get('num_workers', 4),
        pin_memory=True
```

```
    )

    # Save scaler for inference
    full_dataset.save_scaler('models/artifacts/scaler.joblib')

    return train_loader, val_loader, test_loader
```

### 2.3 Training Loop

```
#!/usr/bin/env python3
# training/trainer.py
"""
Generic training loop with MLflow integration

Usage:
    from training.trainer import Trainer
    trainer = Trainer(model, config)
    trainer.train(train_loader, val_loader)
"""

import torch
import torch.nn as nn
from torch.optim import Adam, AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR, ReduceLROnPlateau
from torch.cuda.amp import GradScaler, autocast
import mlflow
from pathlib import Path
from typing import Dict, Optional, Callable
from tqdm import tqdm
from loguru import logger
import numpy as np


class Trainer:
    """Generic PyTorch training loop with MLflow"""

    def __init__(
        self,
        model: nn.Module,
        config: Dict,
        loss_fn: Callable = None,
        device: str = None
    ):
        self.config = config
        self.model = model

        # Device
        if device is None:
            device = 'cuda' if torch.cuda.is_available() else 'cpu'
        self.device = torch.device(device)
        self.model.to(self.device)

        # Loss function
        self.loss_fn = loss_fn or nn.MSELoss()
```

```
        # Optimizer
        train_config = config['training']
        self.optimizer = AdamW(
            model.parameters(),
            lr=train_config['learning_rate'],
            weight_decay=train_config['weight_decay']
        )

        # Scheduler
        scheduler_type = train_config.get('scheduler', 'cosine')
        if scheduler_type == 'cosine':
            self.scheduler = CosineAnnealingLR(
                self.optimizer,
                T_max=train_config['epochs']
            )
        elif scheduler_type == 'plateau':
            self.scheduler = ReduceLROnPlateau(
                self.optimizer,
                mode='min',
                patience=5
            )
        else:
            self.scheduler = None

        # Mixed precision
        self.use_amp = config.get('hardware', {}).get('mixed_precision', False)
        self.scaler = GradScaler() if self.use_amp else None

        # Early stopping
        self.early_stopping = EarlyStopping(
            **train_config.get('early_stopping', {})
        )

        # MLflow setup
        mlflow_config = config.get('mlflow', {})
        if mlflow_config.get('tracking_uri'):
            mlflow.set_tracking_uri(mlflow_config['tracking_uri'])
        mlflow.set_experiment(mlflow_config.get('experiment_name', 'default'))

        # Best model tracking
        self.best_val_loss = float('inf')
        self.best_model_path = None

    def train_epoch(self, train_loader) -> float:
        """Train for one epoch"""
        self.model.train()
        total_loss = 0.0

        pbar = tqdm(train_loader, desc="Training")
        for batch in pbar:
            # Handle both labeled and unlabeled data
            if isinstance(batch, (tuple, list)):
                x, y = batch[0].to(self.device), batch[1].to(self.device)
```

```
            else:
                x = batch.to(self.device)
                y = x  # For autoencoder

            self.optimizer.zero_grad()

            # Forward pass with optional AMP
            if self.use_amp:
                with autocast():
                    output = self.model(x)
                    if isinstance(output, tuple):
                        output = output[0]
                    loss = self.loss_fn(output, y)

                self.scaler.scale(loss).backward()
                self.scaler.step(self.optimizer)
                self.scaler.update()
            else:
                output = self.model(x)
                if isinstance(output, tuple):
                    output = output[0]
                loss = self.loss_fn(output, y)

                loss.backward()
                self.optimizer.step()

            total_loss += loss.item()
            pbar.set_postfix({'loss': loss.item()})

        return total_loss / len(train_loader)

    @torch.no_grad()
    def validate(self, val_loader) -> Dict[str, float]:
        """Validate model"""
        self.model.eval()
        total_loss = 0.0

        for batch in val_loader:
            if isinstance(batch, (tuple, list)):
                x, y = batch[0].to(self.device), batch[1].to(self.device)
            else:
                x = batch.to(self.device)
                y = x

            output = self.model(x)
            if isinstance(output, tuple):
                output = output[0]

            loss = self.loss_fn(output, y)
            total_loss += loss.item()

        val_loss = total_loss / len(val_loader)

        return {'val_loss': val_loss}
```

```python
def train(
    self,
    train_loader,
    val_loader,
    epochs: int = None
):
    """Full training loop"""
    epochs = epochs or self.config['training']['epochs']

    with mlflow.start_run():
        # Log config
        mlflow.log_params(self._flatten_config(self.config))

        for epoch in range(epochs):
            logger.info(f"Epoch {epoch + 1}/{epochs}")

            # Train
            train_loss = self.train_epoch(train_loader)

            # Validate
            val_metrics = self.validate(val_loader)
            val_loss = val_metrics['val_loss']

            # Scheduler step
            if self.scheduler:
                if isinstance(self.scheduler, ReduceLROnPlateau):
                    self.scheduler.step(val_loss)
                else:
                    self.scheduler.step()

            # Log metrics
            mlflow.log_metrics({
                'train_loss': train_loss,
                'val_loss': val_loss,
                'learning_rate': self.optimizer.param_groups[0]['lr']
            }, step=epoch)

            logger.info(f"Train Loss: {train_loss:.6f}, Val Loss: {val_loss:.6f}")

            # Save best model
            if val_loss < self.best_val_loss:
                self.best_val_loss = val_loss
                self.best_model_path = self._save_checkpoint(epoch, val_loss)
                mlflow.log_artifact(self.best_model_path)

            # Early stopping
            if self.early_stopping(val_loss):
                logger.info(f"Early stopping at epoch {epoch + 1}")
                break

        # Log final model
        mlflow.pytorch.log_model(self.model, "model")
```

```
            logger.info(f"Training complete. Best val loss: {self.best_val_loss:.6f}")

    def _save_checkpoint(self, epoch: int, val_loss: float) -> str:
        """Save model checkpoint"""
        checkpoint_dir = Path('models/checkpoints')
        checkpoint_dir.mkdir(parents=True, exist_ok=True)

        path = checkpoint_dir / f"best_model.pt"

        torch.save({
            'epoch': epoch,
            'model_state_dict': self.model.state_dict(),
            'optimizer_state_dict': self.optimizer.state_dict(),
            'val_loss': val_loss,
            'config': self.config
        }, path)

        return str(path)

    def load_checkpoint(self, path: str):
        """Load model checkpoint"""
        checkpoint = torch.load(path, map_location=self.device)
        self.model.load_state_dict(checkpoint['model_state_dict'])
        self.optimizer.load_state_dict(checkpoint['optimizer_state_dict'])

    def _flatten_config(self, config: Dict, parent_key: str = '') -> Dict:
        """Flatten nested config for MLflow"""
        items = {}
        for key, value in config.items():
            new_key = f"{parent_key}.{key}" if parent_key else key
            if isinstance(value, dict):
                items.update(self._flatten_config(value, new_key))
            else:
                items[new_key] = value
        return items


class EarlyStopping:
    """Early stopping to prevent overfitting"""

    def __init__(
        self,
        patience: int = 10,
        min_delta: float = 0.0001,
        mode: str = 'min'
    ):
        self.patience = patience
        self.min_delta = min_delta
        self.mode = mode
        self.counter = 0
        self.best_score = None
        self.early_stop = False

    def __call__(self, score: float) -> bool:
```

```
        if self.best_score is None:
            self.best_score = score
        elif self._is_improvement(score):
            self.best_score = score
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True

        return self.early_stop

    def _is_improvement(self, score: float) -> bool:
        if self.mode == 'min':
            return score < self.best_score - self.min_delta
        else:
            return score > self.best_score + self.min_delta
```

---

# 3. MLFLOW EXPERIMENT TRACKING

## 3.1 MLflow Setup

```
# Install MLflow
pip install mlflow

# Start MLflow server
mlflow server \
    --backend-store-uri postgresql://mlflow:password@postgres:5432/mlflow \
    --default-artifact-root s3://mlflow-artifacts/ \
    --host 0.0.0.0 \
    --port 5000

# Set tracking URI
export MLFLOW_TRACKING_URI="http://mlflow.oilgas.internal:5000"
```

## 3.2 MLflow CLI Commands

```
# List experiments
mlflow experiments search

# Create experiment
mlflow experiments create --experiment-name "anomaly_detection_v2"

# List runs
mlflow runs search --experiment-id 1

# Download artifacts
mlflow artifacts download --run-id <run_id> --artifact-path model

# Serve model
mlflow models serve -m "runs:/<run_id>/model" --port 5001
```

```
# Build docker image
mlflow models build-docker -m "runs:/<run_id>/model" -n "oilgas-anomaly-detector"
```

### 3.3 MLflow Python API

```
#!/usr/bin/env python3
# utils/mlflow_utils.py
"""
MLflow utility functions

Usage:
    from utils.mlflow_utils import MLflowLogger
    logger = MLflowLogger("anomaly_detection")
    with logger.start_run("experiment_1"):
        logger.log_params(config)
        logger.log_metrics({"accuracy": 0.95})
"""

import mlflow
from mlflow.tracking import MlflowClient
from pathlib import Path
from typing import Dict, Any, Optional
from loguru import logger


class MLflowLogger:
    """MLflow experiment tracking wrapper"""

    def __init__(
        self,
        experiment_name: str,
        tracking_uri: str = None
    ):
        if tracking_uri:
            mlflow.set_tracking_uri(tracking_uri)

        self.experiment_name = experiment_name
        mlflow.set_experiment(experiment_name)

        self.client = MlflowClient()
        self.run = None

    def start_run(self, run_name: str = None, tags: Dict = None):
        """Start MLflow run (context manager)"""
        return mlflow.start_run(run_name=run_name, tags=tags)

    def log_params(self, params: Dict[str, Any]):
        """Log parameters"""
        mlflow.log_params(params)

    def log_metrics(self, metrics: Dict[str, float], step: int = None):
        """Log metrics"""
        mlflow.log_metrics(metrics, step=step)
```

```
def log_artifact(self, local_path: str, artifact_path: str = None):
    """Log artifact file"""
    mlflow.log_artifact(local_path, artifact_path)

def log_model(
    self,
    model,
    artifact_path: str = "model",
    registered_model_name: str = None
):
    """Log PyTorch model"""
    mlflow.pytorch.log_model(
        model,
        artifact_path,
        registered_model_name=registered_model_name
    )

def log_figure(self, figure, artifact_file: str):
    """Log matplotlib figure"""
    mlflow.log_figure(figure, artifact_file)

def load_model(self, run_id: str, artifact_path: str = "model"):
    """Load model from run"""
    model_uri = f"runs:/{run_id}/{artifact_path}"
    return mlflow.pytorch.load_model(model_uri)

def get_best_run(self, metric: str = "val_loss", ascending: bool = True):
    """Get best run by metric"""
    experiment = mlflow.get_experiment_by_name(self.experiment_name)

    runs = self.client.search_runs(
        experiment_ids=[experiment.experiment_id],
        order_by=[f"metrics.{metric} {'ASC' if ascending else 'DESC'}"],
        max_results=1
    )

    if runs:
        return runs[0]
    return None

def register_model(
    self,
    run_id: str,
    model_name: str,
    artifact_path: str = "model"
) -> str:
    """Register model to Model Registry"""
    model_uri = f"runs:/{run_id}/{artifact_path}"

    result = mlflow.register_model(model_uri, model_name)
    logger.info(f"Registered model {model_name} version {result.version}")

    return result.version
```

```
    def transition_model_stage(
        self,
        model_name: str,
        version: str,
        stage: str  # "Staging", "Production", "Archived"
    ):
        """Transition model to stage"""
        self.client.transition_model_version_stage(
            name=model_name,
            version=version,
            stage=stage
        )
        logger.info(f"Transitioned {model_name} v{version} to {stage}")

    def load_production_model(self, model_name: str):
        """Load model from Production stage"""
        model_uri = f"models:/{model_name}/Production"
        return mlflow.pytorch.load_model(model_uri)


# Experiment comparison
def compare_experiments(
    experiment_name: str,
    metric: str = "val_loss",
    top_n: int = 10
) -> list:
    """Compare top runs in experiment"""
    client = MlflowClient()
    experiment = mlflow.get_experiment_by_name(experiment_name)

    runs = client.search_runs(
        experiment_ids=[experiment.experiment_id],
        order_by=[f"metrics.{metric} ASC"],
        max_results=top_n
    )

    results = []
    for run in runs:
        results.append({
            'run_id': run.info.run_id,
            'run_name': run.info.run_name,
            metric: run.data.metrics.get(metric),
            'params': run.data.params
        })

    return results
```

---

# 4. MODEL TRAINING WORKFLOWS

## 4.1 Anomaly Detection Training

```python
#!/usr/bin/env python3
# training/train_anomaly.py
"""
Train anomaly detection model

Usage:
    python training/train_anomaly.py --config configs/anomaly_detection.yaml
    python training/train_anomaly.py --config configs/anomaly_detection.yaml --resume checkpoint.pt
"""

import argparse
import torch
from pathlib import Path

from data.datasets import SensorDataset, create_dataloaders
from models.anomaly.autoencoder import AnomalyAutoencoder
from training.trainer import Trainer
from utils.config import load_config
from utils.mlflow_utils import MLflowLogger
from loguru import logger


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True, help='Config file path')
    parser.add_argument('--resume', help='Checkpoint to resume from')
    args = parser.parse_args()

    # Load config
    config = load_config(args.config)
    logger.info(f"Loaded config from {args.config}")

    # Create dataloaders
    train_loader, val_loader, test_loader = create_dataloaders(
        config,
        dataset_class=SensorDataset
    )

    # Create model
    model_config = config['model']
    model = AnomalyAutoencoder(
        input_dim=model_config['input_dim'],
        latent_dim=model_config['latent_dim'],
        hidden_dims=model_config['hidden_dims'],
        dropout=model_config['dropout']
    )

    logger.info(f"Created model with {sum(p.numel() for p in model.parameters())} parameters")

    # Resume from checkpoint
```

```
    if args.resume:
        checkpoint = torch.load(args.resume)
        model.load_state_dict(checkpoint['model_state_dict'])
        logger.info(f"Resumed from {args.resume}")

    # Create trainer
    trainer = Trainer(model, config)

    # Train
    trainer.train(train_loader, val_loader)

    # Evaluate on test set
    test_metrics = trainer.validate(test_loader)
    logger.info(f"Test metrics: {test_metrics}")

    # Fit threshold on validation data
    model.eval()
    threshold = model.fit_threshold(val_loader, percentile=95.0)
    logger.info(f"Fitted anomaly threshold: {threshold:.6f}")

    # Save final model
    output_dir = Path('models/trained')
    output_dir.mkdir(parents=True, exist_ok=True)

    torch.save({
        'model_state_dict': model.state_dict(),
        'config': config,
        'threshold': threshold,
        'test_metrics': test_metrics
    }, output_dir / 'anomaly_detector_final.pt')

    logger.info(f"Saved final model to {output_dir / 'anomaly_detector_final.pt'}")


if __name__ == '__main__':
    main()
```

### 4.2 Distributed Training

```
#!/usr/bin/env python3
# training/train_distributed.py
"""
Distributed training with PyTorch DDP

Usage:
    torchrun --nproc_per_node=4 training/train_distributed.py --config configs/rul_prediction.yaml
"""

import os
import argparse
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler
```

```python
from data.datasets import SequenceDataset
from models.rul.cnn_lstm import CNNLSTM_RUL
from utils.config import load_config
from loguru import logger


def setup_distributed():
    """Initialize distributed training"""
    dist.init_process_group(backend='nccl')
    local_rank = int(os.environ['LOCAL_RANK'])
    torch.cuda.set_device(local_rank)
    return local_rank


def cleanup():
    """Cleanup distributed"""
    dist.destroy_process_group()


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
    args = parser.parse_args()

    # Setup distributed
    local_rank = setup_distributed()
    world_size = dist.get_world_size()
    is_main = local_rank == 0

    if is_main:
        logger.info(f"Training with {world_size} GPUs")

    # Load config
    config = load_config(args.config)

    # Create dataset
    dataset = SequenceDataset(
        data_path=config['data']['train_path'],
        sequence_length=50,
        fit_scaler=True
    )

    # Distributed sampler
    sampler = DistributedSampler(dataset, num_replicas=world_size, rank=local_rank)

    dataloader = torch.utils.data.DataLoader(
        dataset,
        batch_size=config['data']['batch_size'],
        sampler=sampler,
        num_workers=4,
        pin_memory=True
    )
```

```
    # Create model
    model = CNNLSTM_RUL(**config['model']).cuda(local_rank)
    model = DDP(model, device_ids=[local_rank])

    # Optimizer
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=config['training']['learning_rate']
    )

    # Loss
    criterion = torch.nn.MSELoss()

    # Training loop
    for epoch in range(config['training']['epochs']):
        sampler.set_epoch(epoch)
        model.train()

        total_loss = 0.0
        for batch_idx, (x, y) in enumerate(dataloader):
            x, y = x.cuda(local_rank), y.cuda(local_rank)

            optimizer.zero_grad()
            output = model(x)
            loss = criterion(output.squeeze(), y)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        # Average loss across GPUs
        avg_loss = total_loss / len(dataloader)
        avg_loss_tensor = torch.tensor([avg_loss]).cuda(local_rank)
        dist.all_reduce(avg_loss_tensor, op=dist.ReduceOp.SUM)
        avg_loss = avg_loss_tensor.item() / world_size

        if is_main:
            logger.info(f"Epoch {epoch + 1}: Loss = {avg_loss:.6f}")

    # Save model (only on main process)
    if is_main:
        torch.save(model.module.state_dict(), 'models/trained/rul_model.pt')
        logger.info("Model saved")

    cleanup()


if __name__ == '__main__':
    main()
```

# 5. TENSORRT OPTIMIZATION

## 5.1 TensorRT Installation

```
# Install TensorRT (with pip)
pip install tensorrt

# Or from NVIDIA (recommended for latest)
# Download from https://developer.nvidia.com/tensorrt

# Verify installation
python -c "import tensorrt; print(tensorrt.__version__)"
```

## 5.2 PyTorch to TensorRT Conversion

```
#!/usr/bin/env python3
# export/to_tensorrt.py
"""
Convert PyTorch model to TensorRT

Usage:
    python export/to_tensorrt.py --model models/trained/anomaly_detector.pt --output models/tensorrt/
    python export/to_tensorrt.py --model models/trained/rul_model.pt --output models/tensorrt/ --fp16
"""

import argparse
import torch
import tensorrt as trt
import numpy as np
from pathlib import Path
from loguru import logger

# Import model classes
from models.anomaly.autoencoder import AnomalyAutoencoder
from models.rul.cnn_lstm import CNNLSTM_RUL


def pytorch_to_onnx(
    model: torch.nn.Module,
    input_shape: tuple,
    output_path: str,
    opset_version: int = 17
):
    """Export PyTorch model to ONNX"""
    model.eval()

    # Create dummy input
    dummy_input = torch.randn(*input_shape)

    # Export
    torch.onnx.export(
        model,
        dummy_input,
        output_path,
        opset_version=opset_version,
```

```
        input_names=['input'],
        output_names=['output'],
        dynamic_axes={
            'input': {0: 'batch_size'},
            'output': {0: 'batch_size'}
        }
    )

    logger.info(f"Exported ONNX model to {output_path}")


def onnx_to_tensorrt(
    onnx_path: str,
    output_path: str,
    fp16: bool = False,
    int8: bool = False,
    max_batch_size: int = 32,
    workspace_mb: int = 1024
):
    """Convert ONNX model to TensorRT engine"""
    TRT_LOGGER = trt.Logger(trt.Logger.WARNING)

    # Create builder
    builder = trt.Builder(TRT_LOGGER)
    config = builder.create_builder_config()

    # Set memory
    config.set_memory_pool_limit(trt.MemoryPoolType.WORKSPACE, workspace_mb * (1 << 20))

    # Enable FP16/INT8
    if fp16:
        config.set_flag(trt.BuilderFlag.FP16)
        logger.info("Enabled FP16 precision")

    if int8:
        config.set_flag(trt.BuilderFlag.INT8)
        logger.info("Enabled INT8 precision")

    # Create network
    network = builder.create_network(
        1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
    )

    # Parse ONNX
    parser = trt.OnnxParser(network, TRT_LOGGER)

    with open(onnx_path, 'rb') as f:
        if not parser.parse(f.read()):
            for error in range(parser.num_errors):
                logger.error(parser.get_error(error))
            raise RuntimeError("ONNX parsing failed")

    logger.info(f"Parsed ONNX model from {onnx_path}")
```

```
    # Build engine
    serialized_engine = builder.build_serialized_network(network, config)

    # Save engine
    with open(output_path, 'wb') as f:
        f.write(serialized_engine)

    logger.info(f"Saved TensorRT engine to {output_path}")


class TensorRTInference:
    """TensorRT inference wrapper"""

    def __init__(self, engine_path: str):
        self.logger = trt.Logger(trt.Logger.WARNING)

        # Load engine
        with open(engine_path, 'rb') as f:
            runtime = trt.Runtime(self.logger)
            self.engine = runtime.deserialize_cuda_engine(f.read())

        self.context = self.engine.create_execution_context()

        # Get input/output info
        self.input_name = self.engine.get_tensor_name(0)
        self.output_name = self.engine.get_tensor_name(1)

        self.input_shape = self.engine.get_tensor_shape(self.input_name)
        self.output_shape = self.engine.get_tensor_shape(self.output_name)

        logger.info(f"Loaded TensorRT engine from {engine_path}")
        logger.info(f"Input shape: {self.input_shape}, Output shape: {self.output_shape}")

    def infer(self, input_data: np.ndarray) -> np.ndarray:
        """Run inference"""
        import pycuda.driver as cuda
        import pycuda.autoinit

        batch_size = input_data.shape[0]

        # Allocate device memory
        d_input = cuda.mem_alloc(input_data.nbytes)
        output = np.empty((batch_size, *self.output_shape[1:]), dtype=np.float32)
        d_output = cuda.mem_alloc(output.nbytes)

        # Copy input to device
        cuda.memcpy_htod(d_input, input_data)

        # Set tensor addresses
        self.context.set_tensor_address(self.input_name, int(d_input))
        self.context.set_tensor_address(self.output_name, int(d_output))

        # Run inference
        self.context.execute_async_v3(stream_handle=0)
```

```
        # Copy output to host
        cuda.memcpy_dtoh(output, d_output)

        return output


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model', required=True, help='PyTorch model path')
    parser.add_argument('--output', required=True, help='Output directory')
    parser.add_argument('--fp16', action='store_true', help='Enable FP16')
    parser.add_argument('--int8', action='store_true', help='Enable INT8')
    parser.add_argument('--batch-size', type=int, default=32)

    args = parser.parse_args()

    output_dir = Path(args.output)
    output_dir.mkdir(parents=True, exist_ok=True)

    # Load PyTorch model
    checkpoint = torch.load(args.model, map_location='cpu')
    config = checkpoint['config']

    # Determine model type and create instance
    model_name = config['model']['name']

    if model_name == 'AnomalyAutoencoder':
        model = AnomalyAutoencoder(**{k: v for k, v in config['model'].items() if k != 'name'})
        input_shape = (1, config['model']['input_dim'])
    elif model_name == 'CNNLSTM_RUL':
        model = CNNLSTM_RUL(**{k: v for k, v in config['model'].items() if k != 'name'})
        input_shape = (1, 50, config['model']['input_channels'])
    else:
        raise ValueError(f"Unknown model: {model_name}")

    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval()

    # Export to ONNX
    onnx_path = output_dir / f"{model_name}.onnx"
    pytorch_to_onnx(model, input_shape, str(onnx_path))

    # Convert to TensorRT
    precision = 'fp16' if args.fp16 else ('int8' if args.int8 else 'fp32')
    trt_path = output_dir / f"{model_name}_{precision}.engine"

    onnx_to_tensorrt(
        str(onnx_path),
        str(trt_path),
        fp16=args.fp16,
        int8=args.int8,
        max_batch_size=args.batch_size
    )
```

```
    logger.info("Conversion complete!")


if __name__ == '__main__':
    main()
```

---

# 6. ONNX EXPORT & CONVERSION

## 6.1 ONNX Export

```
#!/usr/bin/env python3
# export/to_onnx.py
"""
Export PyTorch models to ONNX format

Usage:
    python export/to_onnx.py --model models/trained/anomaly_detector.pt --output models/onnx/
"""

import argparse
import torch
import onnx
import onnxruntime as ort
import numpy as np
from pathlib import Path
from loguru import logger


def export_to_onnx(
    model: torch.nn.Module,
    input_shape: tuple,
    output_path: str,
    opset_version: int = 17,
    optimize: bool = True
):
    """
    Export PyTorch model to ONNX.

    Args:
        model: PyTorch model
        input_shape: Input tensor shape (including batch dim)
        output_path: Output ONNX file path
        opset_version: ONNX opset version
        optimize: Whether to optimize the model
    """
    model.eval()

    # Create dummy input
    dummy_input = torch.randn(*input_shape)

    # Export
```

```
    torch.onnx.export(
        model,
        dummy_input,
        output_path,
        export_params=True,
        opset_version=opset_version,
        do_constant_folding=True,
        input_names=['input'],
        output_names=['output'],
        dynamic_axes={
            'input': {0: 'batch_size'},
            'output': {0: 'batch_size'}
        }
    )

    logger.info(f"Exported ONNX model to {output_path}")

    # Verify
    onnx_model = onnx.load(output_path)
    onnx.checker.check_model(onnx_model)
    logger.info("ONNX model verified successfully")

    # Optimize
    if optimize:
        from onnxruntime.transformers import optimizer
        optimized_path = output_path.replace('.onnx', '_optimized.onnx')

        optimized_model = optimizer.optimize_model(
            output_path,
            model_type='bert',  # Use appropriate type
            num_heads=0,
            hidden_size=0
        )
        optimized_model.save_model_to_file(optimized_path)
        logger.info(f"Saved optimized model to {optimized_path}")

    return output_path


def verify_onnx_output(
    pytorch_model: torch.nn.Module,
    onnx_path: str,
    input_shape: tuple,
    rtol: float = 1e-3,
    atol: float = 1e-5
):
    """Verify ONNX output matches PyTorch"""
    pytorch_model.eval()

    # Create test input
    test_input = torch.randn(*input_shape)

    # PyTorch inference
    with torch.no_grad():
```

```
        pytorch_output = pytorch_model(test_input)
        if isinstance(pytorch_output, tuple):
            pytorch_output = pytorch_output[0]

    # ONNX Runtime inference
    ort_session = ort.InferenceSession(onnx_path)
    ort_output = ort_session.run(
        None,
        {'input': test_input.numpy()}
    )[0]

    # Compare
    np.testing.assert_allclose(
        pytorch_output.numpy(),
        ort_output,
        rtol=rtol,
        atol=atol
    )

    logger.info("ONNX output matches PyTorch output!")


class ONNXInference:
    """ONNX Runtime inference wrapper"""

    def __init__(
        self,
        model_path: str,
        providers: list = None
    ):
        if providers is None:
            providers = ['CUDAExecutionProvider', 'CPUExecutionProvider']

        self.session = ort.InferenceSession(model_path, providers=providers)

        self.input_name = self.session.get_inputs()[0].name
        self.output_name = self.session.get_outputs()[0].name

        logger.info(f"Loaded ONNX model from {model_path}")
        logger.info(f"Providers: {self.session.get_providers()}")

    def infer(self, input_data: np.ndarray) -> np.ndarray:
        """Run inference"""
        return self.session.run(
            [self.output_name],
            {self.input_name: input_data.astype(np.float32)}
        )[0]

    def infer_batch(
        self,
        data: np.ndarray,
        batch_size: int = 32
    ) -> np.ndarray:
        """Run batched inference"""
```

```
        results = []

        for i in range(0, len(data), batch_size):
            batch = data[i:i + batch_size]
            output = self.infer(batch)
            results.append(output)

        return np.concatenate(results, axis=0)
```

---

# 7. MODEL EVALUATION TOOLS

## 7.1 Evaluation Script

```
#!/usr/bin/env python3
# evaluation/evaluate.py
"""
Model evaluation tools

Usage:
    python evaluation/evaluate.py --model models/trained/anomaly_detector.pt --data data/test.parquet
"""

import argparse
import torch
import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    mean_absolute_error, mean_squared_error, r2_score
)
from pathlib import Path
import json
from loguru import logger


class ModelEvaluator:
    """Comprehensive model evaluation"""

    def __init__(self, model, device='cuda'):
        self.model = model
        self.device = torch.device(device if torch.cuda.is_available() else 'cpu')
        self.model.to(self.device)
        self.model.eval()

    def evaluate_anomaly_detection(
        self,
        dataloader,
        threshold: float,
        labels: np.ndarray = None
    ) -> dict:
        """Evaluate anomaly detection model"""
```

```
    all_scores = []
    all_predictions = []

    with torch.no_grad():
        for batch in dataloader:
            if isinstance(batch, (tuple, list)):
                x = batch[0].to(self.device)
            else:
                x = batch.to(self.device)

            scores = self.model.compute_anomaly_score(x)
            predictions = (scores > threshold).int()

            all_scores.extend(scores.cpu().numpy())
            all_predictions.extend(predictions.cpu().numpy())

    results = {
        'threshold': threshold,
        'mean_score': float(np.mean(all_scores)),
        'std_score': float(np.std(all_scores)),
        'anomaly_rate': float(np.mean(all_predictions)),
        'total_samples': len(all_scores)
    }

    # If labels available
    if labels is not None:
        results['accuracy'] = accuracy_score(labels, all_predictions)
        results['precision'] = precision_score(labels, all_predictions)
        results['recall'] = recall_score(labels, all_predictions)
        results['f1'] = f1_score(labels, all_predictions)
        results['auc_roc'] = roc_auc_score(labels, all_scores)
        results['confusion_matrix'] = confusion_matrix(labels, all_predictions).tolist()

    return results

def evaluate_rul_prediction(
    self,
    dataloader
) -> dict:
    """Evaluate RUL prediction model"""
    all_predictions = []
    all_targets = []

    with torch.no_grad():
        for x, y in dataloader:
            x = x.to(self.device)
            pred = self.model(x).squeeze().cpu().numpy()
            all_predictions.extend(pred)
            all_targets.extend(y.numpy())

    all_predictions = np.array(all_predictions)
    all_targets = np.array(all_targets)

    # Compute metrics
```

```
        mae = mean_absolute_error(all_targets, all_predictions)
        rmse = np.sqrt(mean_squared_error(all_targets, all_predictions))
        r2 = r2_score(all_targets, all_predictions)

        # Percentage within thresholds
        errors = np.abs(all_predictions - all_targets)
        within_5 = float(np.mean(errors <= 5))
        within_10 = float(np.mean(errors <= 10))
        within_20 = float(np.mean(errors <= 20))

        return {
            'mae': float(mae),
            'rmse': float(rmse),
            'r2': float(r2),
            'within_5_days': within_5,
            'within_10_days': within_10,
            'within_20_days': within_20,
            'mean_prediction': float(np.mean(all_predictions)),
            'mean_target': float(np.mean(all_targets)),
            'total_samples': len(all_targets)
        }

    def compute_latency(
        self,
        input_shape: tuple,
        n_iterations: int = 100,
        warmup: int = 10
    ) -> dict:
        """Measure inference latency"""
        dummy_input = torch.randn(*input_shape).to(self.device)

        # Warmup
        for _ in range(warmup):
            with torch.no_grad():
                _ = self.model(dummy_input)

        # Synchronize
        if self.device.type == 'cuda':
            torch.cuda.synchronize()

        # Measure
        import time
        latencies = []

        for _ in range(n_iterations):
            start = time.perf_counter()

            with torch.no_grad():
                _ = self.model(dummy_input)

            if self.device.type == 'cuda':
                torch.cuda.synchronize()

            latencies.append((time.perf_counter() - start) * 1000)  # ms
```

```
        return {
            'mean_latency_ms': float(np.mean(latencies)),
            'std_latency_ms': float(np.std(latencies)),
            'p50_latency_ms': float(np.percentile(latencies, 50)),
            'p95_latency_ms': float(np.percentile(latencies, 95)),
            'p99_latency_ms': float(np.percentile(latencies, 99)),
            'throughput_samples_per_sec': float(1000 / np.mean(latencies))
        }

    def compute_model_size(self) -> dict:
        """Compute model size metrics"""
        # Parameter count
        total_params = sum(p.numel() for p in self.model.parameters())
        trainable_params = sum(p.numel() for p in self.model.parameters() if p.requires_grad)

        # Memory footprint
        param_size = sum(p.numel() * p.element_size() for p in self.model.parameters())
        buffer_size = sum(b.numel() * b.element_size() for b in self.model.buffers())
        total_size = param_size + buffer_size

        return {
            'total_parameters': total_params,
            'trainable_parameters': trainable_params,
            'model_size_mb': total_size / (1024 * 1024),
            'param_size_mb': param_size / (1024 * 1024)
        }


def generate_evaluation_report(
    results: dict,
    output_path: str
):
    """Generate evaluation report"""
    report = {
        'evaluation_timestamp': pd.Timestamp.now().isoformat(),
        'results': results
    }

    # Save JSON
    with open(output_path, 'w') as f:
        json.dump(report, f, indent=2)

    logger.info(f"Saved evaluation report to {output_path}")

    # Print summary
    print("\n" + "=" * 50)
    print("MODEL EVALUATION REPORT")
    print("=" * 50)

    for category, metrics in results.items():
        print(f"\n{category.upper()}:")
        if isinstance(metrics, dict):
            for key, value in metrics.items():
```

```
            if isinstance(value, float):
                print(f"  {key}: {value:.4f}")
            else:
                print(f"  {key}: {value}")
```

---

# 8. RAG PIPELINE TOOLS

## 8.1 Embedding Generation

```
#!/usr/bin/env python3
# models/rag/embeddings.py
"""
Embedding generation for RAG pipeline

Usage:
    from models.rag.embeddings import EmbeddingModel
    embedder = EmbeddingModel("BAAI/bge-large-en-v1.5")
    embeddings = embedder.encode(texts)
"""

import torch
import numpy as np
from typing import List, Union
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModel
from loguru import logger


class EmbeddingModel:
    """Text embedding model for RAG"""

    def __init__(
        self,
        model_name: str = "BAAI/bge-large-en-v1.5",
        device: str = None,
        normalize: bool = True
    ):
        self.model_name = model_name
        self.normalize = normalize

        if device is None:
            device = 'cuda' if torch.cuda.is_available() else 'cpu'
        self.device = device

        # Load model
        self.model = SentenceTransformer(model_name, device=device)
        self.dim = self.model.get_sentence_embedding_dimension()

        logger.info(f"Loaded {model_name} (dim={self.dim}) on {device}")

    def encode(
        self,
```

```
        texts: Union[str, List[str]],
        batch_size: int = 32,
        show_progress: bool = False
    ) -> np.ndarray:
        """
        Encode texts to embeddings.

        Args:
            texts: Single text or list of texts
            batch_size: Batch size for encoding
            show_progress: Show progress bar

        Returns:
            Embeddings array [n_texts, dim]
        """
        if isinstance(texts, str):
            texts = [texts]

        embeddings = self.model.encode(
            texts,
            batch_size=batch_size,
            show_progress_bar=show_progress,
            normalize_embeddings=self.normalize
        )

        return embeddings

    def encode_query(self, query: str) -> np.ndarray:
        """Encode query with instruction prefix (for asymmetric models)"""
        # BGE models use instruction prefix for queries
        if 'bge' in self.model_name.lower():
            query = f"Represent this sentence for searching relevant passages: {query}"

        return self.encode(query)[0]

    def encode_documents(
        self,
        documents: List[str],
        batch_size: int = 32
    ) -> np.ndarray:
        """Encode documents"""
        return self.encode(documents, batch_size=batch_size, show_progress=True)


class HuggingFaceEmbeddings:
    """Alternative embedding model using HuggingFace directly"""

    def __init__(
        self,
        model_name: str = "BAAI/bge-large-en-v1.5",
        device: str = None
    ):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)
```

```
        if device is None:
            device = 'cuda' if torch.cuda.is_available() else 'cpu'
        self.device = device
        self.model.to(device)
        self.model.eval()

    def mean_pooling(self, model_output, attention_mask):
        """Mean pooling of token embeddings"""
        token_embeddings = model_output[0]
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(token_embeddings.size()).float()
        return torch.sum(token_embeddings * input_mask_expanded, 1) / torch.clamp(input_mask_expanded.su

    def encode(self, texts: List[str], batch_size: int = 32) -> np.ndarray:
        """Encode texts"""
        all_embeddings = []

        for i in range(0, len(texts), batch_size):
            batch = texts[i:i + batch_size]

            encoded = self.tokenizer(
                batch,
                padding=True,
                truncation=True,
                max_length=512,
                return_tensors='pt'
            )
            encoded = {k: v.to(self.device) for k, v in encoded.items()}

            with torch.no_grad():
                output = self.model(**encoded)
                embeddings = self.mean_pooling(output, encoded['attention_mask'])
                embeddings = torch.nn.functional.normalize(embeddings, p=2, dim=1)

            all_embeddings.append(embeddings.cpu().numpy())

        return np.concatenate(all_embeddings, axis=0)
```

## 8.2 RAG Pipeline

```
#!/usr/bin/env python3
# models/rag/pipeline.py
"""
Complete RAG pipeline

Usage:
    from models.rag.pipeline import RAGPipeline
    rag = RAGPipeline()
    answer = rag.query("What is the maintenance schedule for PUMP-001?")
"""

import os
from typing import List, Dict, Optional
import numpy as np
```

```python
from loguru import logger

from models.rag.embeddings import EmbeddingModel
# Assume milvus_client is available
# from scripts.milvus_client import MilvusClient


class RAGPipeline:
    """Retrieval-Augmented Generation pipeline"""

    def __init__(
        self,
        embedding_model: str = "BAAI/bge-large-en-v1.5",
        collection_name: str = "document_embeddings",
        llm_model: str = "gpt-4",
        top_k_retrieve: int = 20,
        top_k_rerank: int = 5
    ):
        # Embedding model
        self.embedder = EmbeddingModel(embedding_model)

        # Vector database
        # self.vector_db = MilvusClient()
        self.collection_name = collection_name

        # LLM settings
        self.llm_model = llm_model

        # Retrieval settings
        self.top_k_retrieve = top_k_retrieve
        self.top_k_rerank = top_k_rerank

        logger.info("Initialized RAG pipeline")

    def retrieve(
        self,
        query: str,
        filter_expr: str = None
    ) -> List[Dict]:
        """
        Retrieve relevant documents.

        Args:
            query: User query
            filter_expr: Optional Milvus filter expression

        Returns:
            List of retrieved documents with scores
        """
        # Encode query
        query_embedding = self.embedder.encode_query(query)

        # Vector search
        # results = self.vector_db.search_similar(
```

```
        #     self.collection_name,
        #     query_embedding.tolist(),
        #     limit=self.top_k_retrieve,
        #     filter_expr=filter_expr
        # )

        # Placeholder for demo
        results = []

        return results

    def rerank(
        self,
        query: str,
        documents: List[Dict],
        top_k: int = None
    ) -> List[Dict]:
        """
        Rerank retrieved documents.

        Uses cross-encoder for more accurate relevance scoring.
        """
        top_k = top_k or self.top_k_rerank

        # Simple score-based reranking (replace with cross-encoder in production)
        sorted_docs = sorted(documents, key=lambda x: x['score'], reverse=True)

        return sorted_docs[:top_k]

    def generate_context(self, documents: List[Dict]) -> str:
        """Combine documents into context string"""
        context_parts = []

        for i, doc in enumerate(documents, 1):
            context_parts.append(f"[Document {i}]\n{doc['text']}\n")

        return "\n".join(context_parts)

    def generate_answer(
        self,
        query: str,
        context: str
    ) -> str:
        """
        Generate answer using LLM.

        Args:
            query: User query
            context: Retrieved context

        Returns:
            Generated answer
        """
        # Build prompt
```

```
        prompt = f"""Based on the following context, answer the question.

Context:
{context}

Question: {query}

Answer:"""

        # Call LLM (using OpenAI as example)
        import openai

        response = openai.ChatCompletion.create(
            model=self.llm_model,
            messages=[
                {"role": "system", "content": "You are a helpful assistant for oil and gas operations. A
                {"role": "user", "content": prompt}
            ],
            temperature=0.1,
            max_tokens=500
        )

        return response.choices[0].message.content

    def query(
        self,
        query: str,
        filter_expr: str = None,
        return_sources: bool = True
    ) -> Dict:
        """
        Full RAG query pipeline.

        Args:
            query: User question
            filter_expr: Optional document filter
            return_sources: Include source documents in response

        Returns:
            Dict with answer and optionally sources
        """
        logger.info(f"RAG query: {query}")

        # Step 1: Retrieve
        documents = self.retrieve(query, filter_expr)

        if not documents:
            return {
                'answer': "I couldn't find relevant information to answer your question.",
                'sources': []
            }

        # Step 2: Rerank
        reranked = self.rerank(query, documents)
```

```
        # Step 3: Generate context
        context = self.generate_context(reranked)

        # Step 4: Generate answer
        answer = self.generate_answer(query, context)

        result = {'answer': answer}

        if return_sources:
            result['sources'] = [
                {
                    'document_id': doc.get('document_id'),
                    'chunk_id': doc.get('chunk_id'),
                    'score': doc.get('score'),
                    'text': doc.get('text')[:200] + '...'
                }
                for doc in reranked
            ]

        return result

    def ingest_document(
        self,
        document_id: str,
        text: str,
        metadata: Dict = None,
        chunk_size: int = 512,
        chunk_overlap: int = 50
    ):
        """
        Ingest document into RAG system.

        Args:
            document_id: Unique document ID
            text: Document text
            metadata: Optional metadata
            chunk_size: Characters per chunk
            chunk_overlap: Overlap between chunks
        """
        # Chunk document
        chunks = self._chunk_text(text, chunk_size, chunk_overlap)

        # Generate embeddings
        embeddings = self.embedder.encode_documents([c['text'] for c in chunks])

        # Prepare for insertion
        for i, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
            chunk['chunk_id'] = f"{document_id}_chunk_{i}"
            chunk['embedding'] = embedding.tolist()
            chunk['metadata'] = metadata or {}

        # Insert to vector DB
        # self.vector_db.insert_document_chunks(
```

```
        #     self.collection_name,
        #     document_id,
        #     chunks
        # )

        logger.info(f"Ingested document {document_id} ({len(chunks)} chunks)")

    def _chunk_text(
        self,
        text: str,
        chunk_size: int,
        overlap: int
    ) -> List[Dict]:
        """Split text into overlapping chunks"""
        chunks = []
        start = 0

        while start < len(text):
            end = start + chunk_size
            chunk_text = text[start:end]

            # Try to break at sentence boundary
            if end < len(text):
                last_period = chunk_text.rfind('.')
                if last_period > chunk_size * 0.5:
                    chunk_text = chunk_text[:last_period + 1]
                    end = start + last_period + 1

            chunks.append({
                'text': chunk_text,
                'start_char': start,
                'end_char': end
            })

            start = end - overlap

        return chunks
```

---

# 9. MODEL DEPLOYMENT

## 9.1 FastAPI Model Server

```
#!/usr/bin/env python3
# inference/serve.py
"""
FastAPI model serving

Usage:
    uvicorn inference.serve:app --host 0.0.0.0 --port 8000
"""

from fastapi import FastAPI, HTTPException
```

```python
from pydantic import BaseModel
from typing import List, Dict, Optional
import torch
import numpy as np
from loguru import logger

from models.anomaly.autoencoder import AnomalyAutoencoder
from models.rul.cnn_lstm import CNNLSTM_RUL


app = FastAPI(title="Oil & Gas ML API")


# Request/Response models
class AnomalyRequest(BaseModel):
    features: List[List[float]]  # [batch_size, num_features]


class AnomalyResponse(BaseModel):
    is_anomaly: List[bool]
    scores: List[float]


class RULRequest(BaseModel):
    sequences: List[List[List[float]]]  # [batch_size, seq_len, features]


class RULResponse(BaseModel):
    rul_predictions: List[float]
    uncertainty: List[float]


class HealthResponse(BaseModel):
    status: str
    models_loaded: Dict[str, bool]


# Model registry
models = {}


@app.on_event("startup")
async def load_models():
    """Load models on startup"""
    device = 'cuda' if torch.cuda.is_available() else 'cpu'

    try:
        # Load anomaly detector
        checkpoint = torch.load('models/trained/anomaly_detector_final.pt', map_location=device)
        anomaly_model = AnomalyAutoencoder(**{
            k: v for k, v in checkpoint['config']['model'].items() if k != 'name'
        })
        anomaly_model.load_state_dict(checkpoint['model_state_dict'])
        anomaly_model.to(device)
```

```
            anomaly_model.eval()

            models['anomaly'] = {
                'model': anomaly_model,
                'threshold': checkpoint.get('threshold', 0.5),
                'device': device
            }
            logger.info("Loaded anomaly detection model")
        except Exception as e:
            logger.error(f"Failed to load anomaly model: {e}")

        try:
            # Load RUL predictor
            checkpoint = torch.load('models/trained/rul_model.pt', map_location=device)
            rul_model = CNNLSTM_RUL(**checkpoint['config']['model'])
            rul_model.load_state_dict(checkpoint['model_state_dict'])
            rul_model.to(device)
            rul_model.eval()

            models['rul'] = {
                'model': rul_model,
                'device': device
            }
            logger.info("Loaded RUL prediction model")
        except Exception as e:
            logger.error(f"Failed to load RUL model: {e}")


    @app.get("/health", response_model=HealthResponse)
    async def health_check():
        """Health check endpoint"""
        return HealthResponse(
            status="healthy",
            models_loaded={
                'anomaly': 'anomaly' in models,
                'rul': 'rul' in models
            }
        )


    @app.post("/predict/anomaly", response_model=AnomalyResponse)
    async def predict_anomaly(request: AnomalyRequest):
        """Anomaly detection endpoint"""
        if 'anomaly' not in models:
            raise HTTPException(status_code=503, detail="Anomaly model not loaded")

        model_info = models['anomaly']
        model = model_info['model']
        threshold = model_info['threshold']
        device = model_info['device']

        # Convert to tensor
        features = torch.tensor(request.features, dtype=torch.float32).to(device)
```

```
    # Inference
    with torch.no_grad():
        is_anomaly, scores = model.predict_anomaly(features, threshold)

    return AnomalyResponse(
        is_anomaly=is_anomaly.cpu().tolist(),
        scores=scores.cpu().tolist()
    )


@app.post("/predict/rul", response_model=RULResponse)
async def predict_rul(request: RULRequest):
    """RUL prediction endpoint"""
    if 'rul' not in models:
        raise HTTPException(status_code=503, detail="RUL model not loaded")

    model_info = models['rul']
    model = model_info['model']
    device = model_info['device']

    # Convert to tensor
    sequences = torch.tensor(request.sequences, dtype=torch.float32).to(device)

    # Inference with uncertainty
    with torch.no_grad():
        mean, std = model.predict_with_uncertainty(sequences, n_samples=50)

    return RULResponse(
        rul_predictions=mean.squeeze().cpu().tolist(),
        uncertainty=std.squeeze().cpu().tolist()
    )


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

---

# 10. COMPLETE ML WORKFLOWS

## 10.1 End-to-End Training Pipeline

```
#!/bin/bash
# workflows/train_all_models.sh
# Complete training pipeline

set -e

echo "ML Training Pipeline"
echo "===================="

# 1. Prepare training data
echo "Step 1: Preparing training data..."
```

```
python scripts/unified_export.py \
    --all-assets \
    --days 90 \
    --output data/training/ \
    --training-dataset

# 2. Train anomaly detection model
echo "Step 2: Training anomaly detection model..."
python training/train_anomaly.py --config configs/anomaly_detection.yaml

# 3. Train RUL prediction model
echo "Step 3: Training RUL prediction model..."
python training/train_rul.py --config configs/rul_prediction.yaml

# 4. Evaluate models
echo "Step 4: Evaluating models..."
python evaluation/evaluate.py \
    --model models/trained/anomaly_detector_final.pt \
    --data data/test.parquet \
    --output evaluation/anomaly_report.json

# 5. Export to ONNX
echo "Step 5: Exporting to ONNX..."
python export/to_onnx.py \
    --model models/trained/anomaly_detector_final.pt \
    --output models/onnx/

# 6. Convert to TensorRT
echo "Step 6: Converting to TensorRT..."
python export/to_tensorrt.py \
    --model models/trained/anomaly_detector_final.pt \
    --output models/tensorrt/ \
    --fp16

# 7. Register models in MLflow
echo "Step 7: Registering models..."
python scripts/register_models.py

echo ""
echo "Training pipeline complete!"
```

### 10.2 Quick Reference

```
# ============================================
# ML TOOLS QUICK REFERENCE
# ============================================

# --- PYTORCH ---
# Train model
python training/train_anomaly.py --config configs/anomaly_detection.yaml

# Distributed training
torchrun --nproc_per_node=4 training/train_distributed.py --config configs/rul.yaml
```

```
# --- MLFLOW ---
# Start server
mlflow server --host 0.0.0.0 --port 5000

# List experiments
mlflow experiments search

# Serve model
mlflow models serve -m "runs:/<run_id>/model" --port 5001

# --- ONNX ---
# Export model
python export/to_onnx.py --model model.pt --output model.onnx

# Verify ONNX
python -c "import onnx; onnx.checker.check_model('model.onnx')"

# --- TENSORRT ---
# Convert to TensorRT
python export/to_tensorrt.py --model model.pt --output model.engine --fp16

# --- INFERENCE ---
# Start API server
uvicorn inference.serve:app --host 0.0.0.0 --port 8000

# Test prediction
curl -X POST "http://localhost:8000/predict/anomaly" \
    -H "Content-Type: application/json" \
    -d '{"features": [[1.0, 2.0, 3.0]]}'

# --- EVALUATION ---
python evaluation/evaluate.py --model model.pt --data test.parquet
```

---

*END OF ML/AI TOOLS & OPERATIONS GUIDE*